(Data) Replication (and Distribution)

Why data replication: failure handling

- ***Fault-tolerance***
  - If failure occurs "failover" to other replica
- ***Availability***
  - Service available as long as client can access one replica
- Difference:
  - Terms not always used clearly

Why Data Replications: Fast Local Access

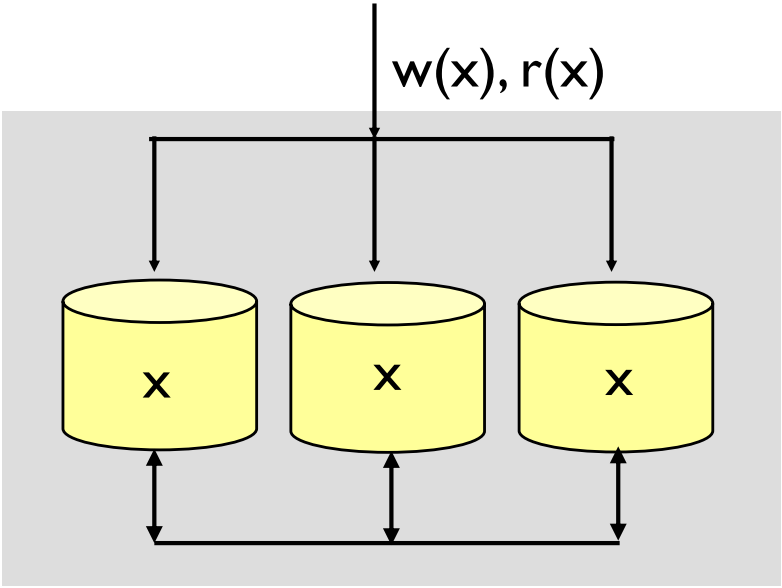
- Fast local access in WAN settings
  - Install data replicas close to clients
    - o Access closest replica
    - Geo-replication

* Disconnected access



Why data replication: Scalability

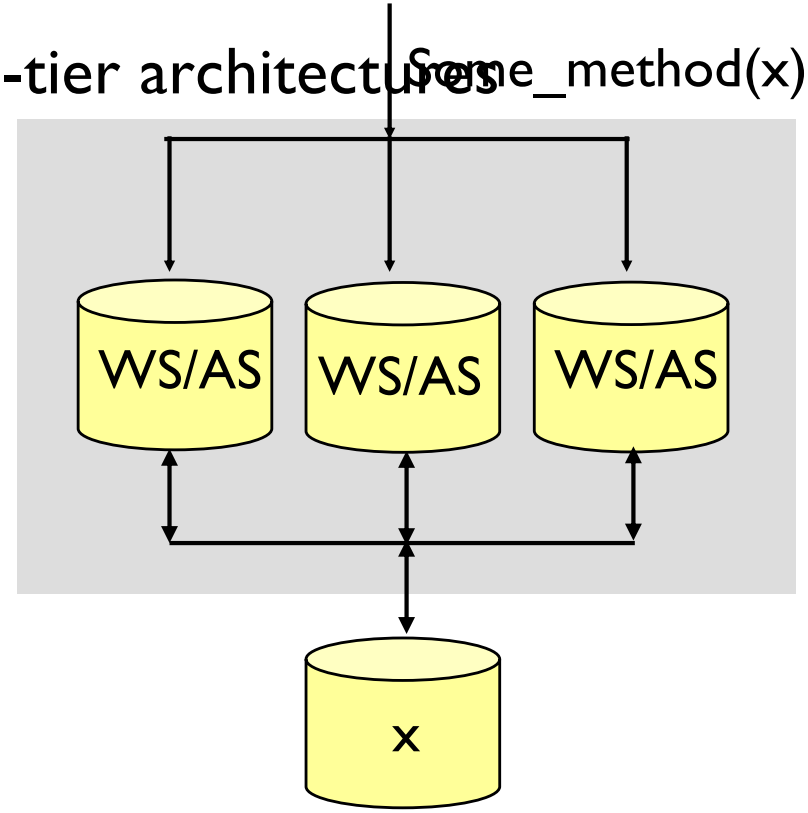
- ***Distribute read requests across data replicas***
  - ROWA (read one / write all)
    - Writes are typically executed at all data replicas and reads at one replica
    - Even update intensive workloads have often a 80 read / 20 write threshold
  - Web-Service Example:
    - Static web-pages replicated
  - ML inference
    - Model replicated
    - Inference on new data can be considered a read of the model data (and some computation)



Replication vs. Distribution
Data vs. Computation

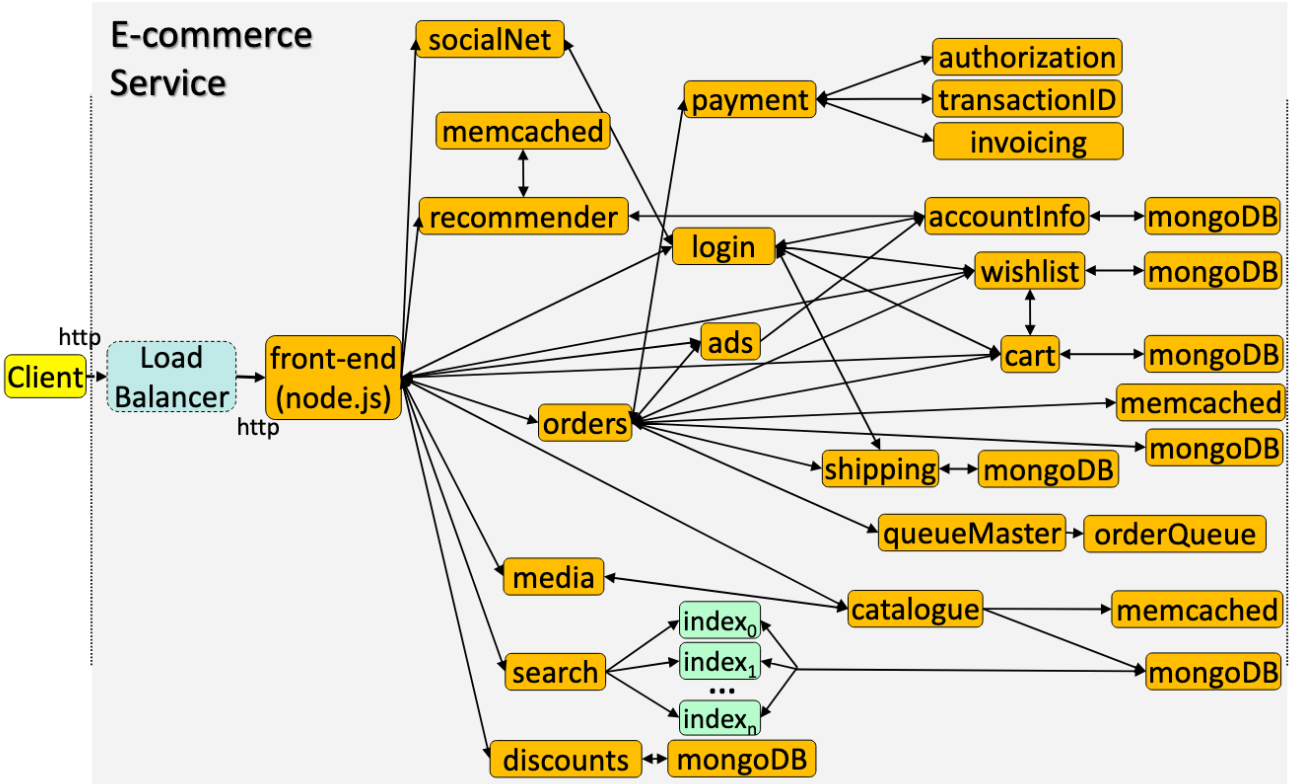
Not data replication: Scalable Service Replication

- ***Execution of one individual request on one of the replicated compute units***
  - Task: load-balancing
  - Web-service / application server replication
  - Data is NOT replicated
  - As there is always "some" data, multi-tier architectures
  - Data managed by data storage engine
    - Possibly replicated for fault-tolerance

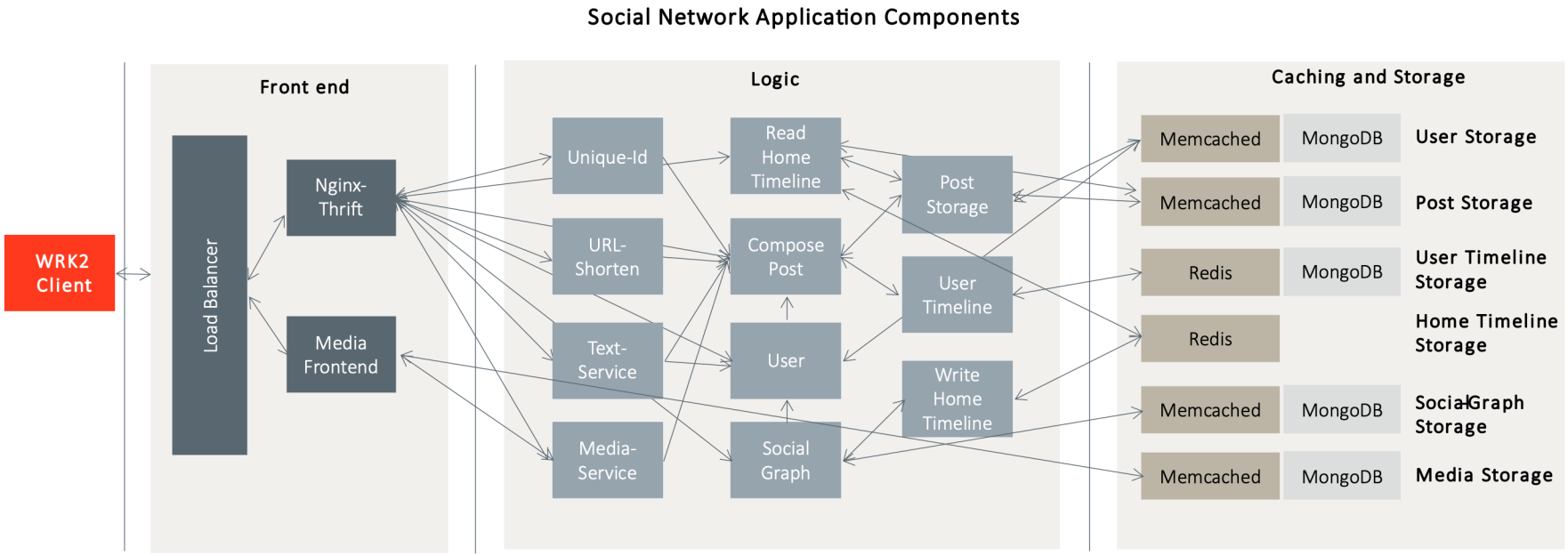


Microservice Architecture

- Split up application into modular components that call each other (micro-services)
- ***Execution of one individual request is split/distributed*** *across* ***distributed compute units***



From: An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems, ASPLOS 2019


Social Network Application Components
Front end
Logic
Caching and Storage
WRK2 Client
Load Balancer
Nginx-Thrift
Media Frontend
Unique-Id
URL-Shorten
Text-Service
Media-Service
Read Home Timeline
Compose Post
User
Social Graph
Post Storage
User Timeline
Write Home Timeline
Memcached
MongoDB
User Storage
Memcached
MongoDB
Post Storage
Redis
MongoDB
User Timeline Storage
Redis
Home Timeline Storage
Memcached
MongoDB
Social Graph Storage
Memcached
MongoDB
Media Storage

Microservice Architecture

- Micro-services can run on same or different nodes
  - Scalability by **distributing tasks** across nodes
- Each micro-service follows request/reply structure
  - Originally, micro-services communicate via http (REST API)
  - Run within a platform (e.g. an application server)
- Individual **micro-services can be replicated** if single node cannot handle load
  - Scalability at the micro-service level
- Important data is managed by backend databases / caches
  - Individual micro-services "stateless"

Distribution and replication of services – data is managed by an extra tier; might be distributed based on micro-service or centralized

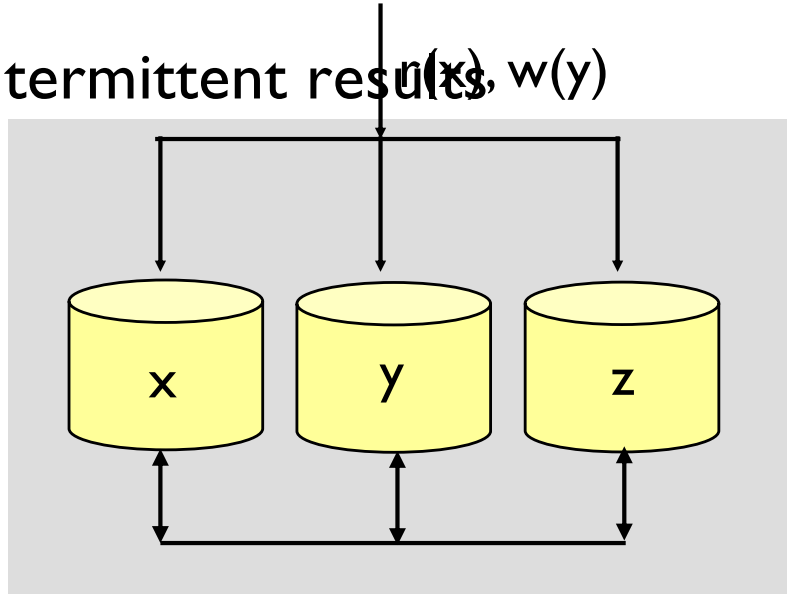
"Serverless computing"

- Offered as functionality by cloud
  - AWS lambda
- Application programmer writes application in form of simple stateless functions
  - Recommender, search, discounts, etc. (similar to micro-service)
- Cloud takes care of executing the functions on some node whenever a client calls them
  - Infrastructure / execution environment managed by the cloud
- Any relevant state must be stored in a backend storage engine (e.g., AWS S3)
  - Functions retrieve state from storage engine / write update backs to storage engine

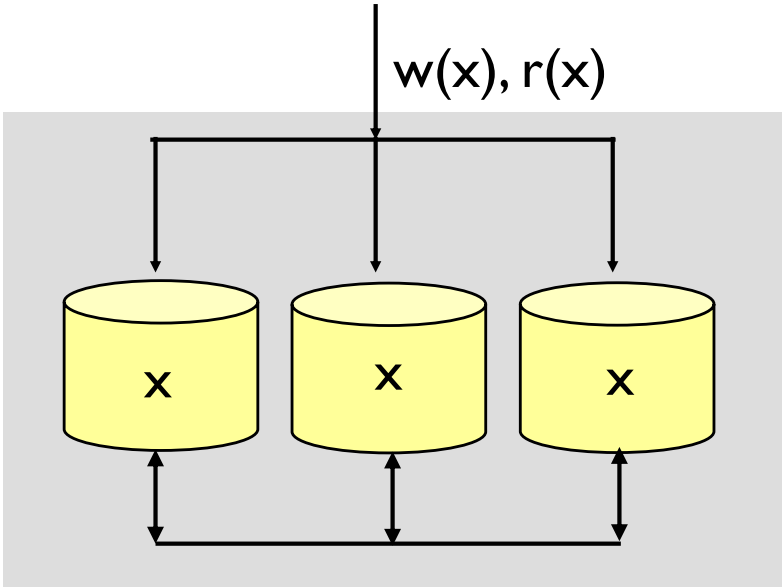
Distribution and replication of functions – data is managed by an extra tier; might be distributed based on functions or centralized

Not data replication: Scalability and **data distribution**

- ***Distribute data across nodes (NO data replication per se)***
- ***Distribute requests to where requested data resides***
  - Challenge: distribute data as to have even load across partitions
  - What if a request wants to access data from 2 partitions?
  - Other challenges?
- ***Distribute computation, each working on local data set***
  - Big data / analytics
  - challenge: how to aggregate intermittent results



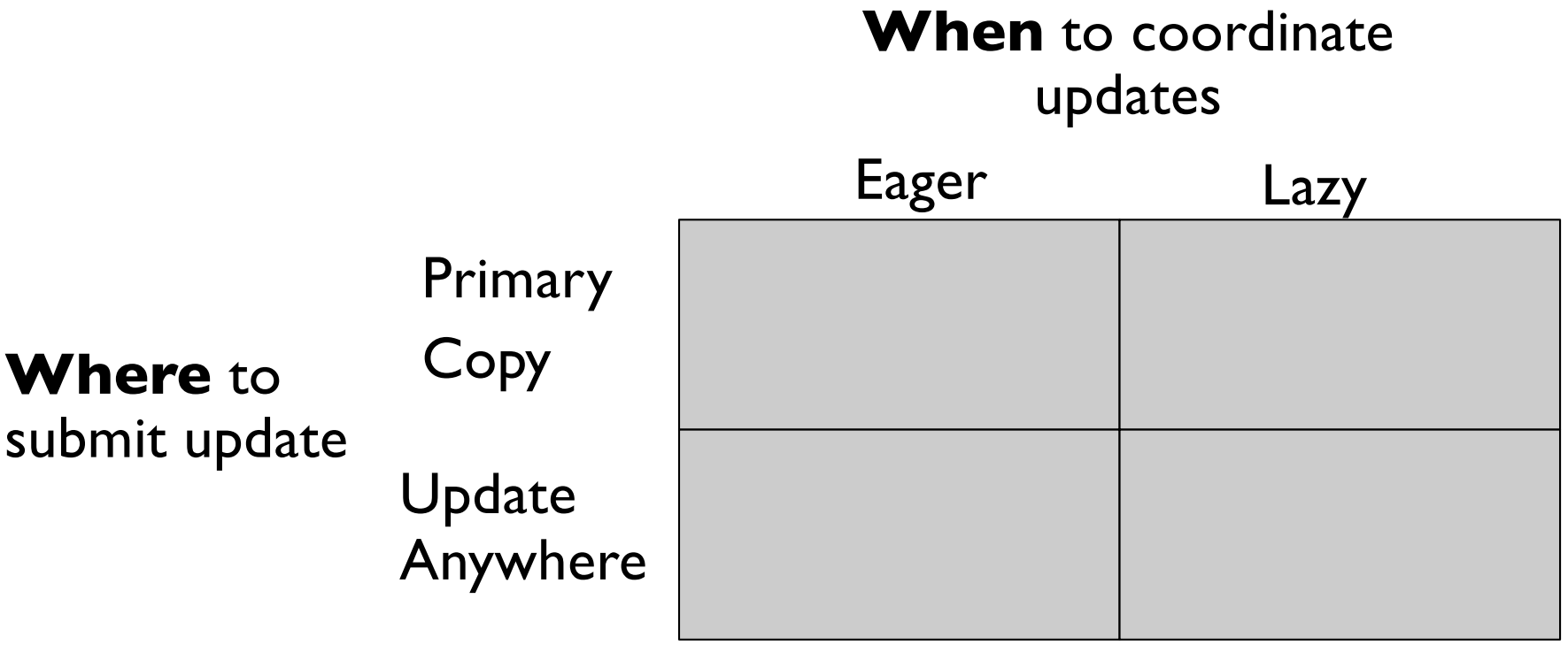
Back to Data Replication….



- Challenges:
  - Load-Distribution of read requests
    - How do you know that a request is a read or a write?
    - How do you know how complex each of the read requests is?
    - How do you keep replicas equally loaded?
  - Data Consistency
    - Keep copies consistent despite concurrent updates
      - (we have seen Paxos so far…)

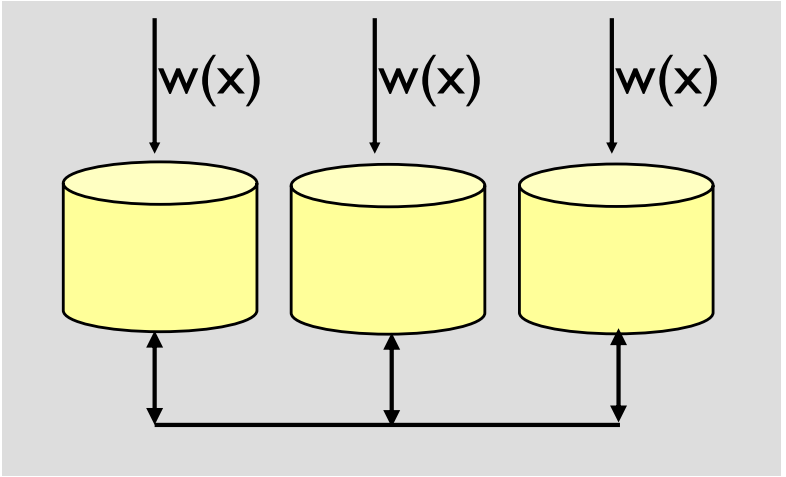
Simple Classification

- ***Controlling consistency of data***

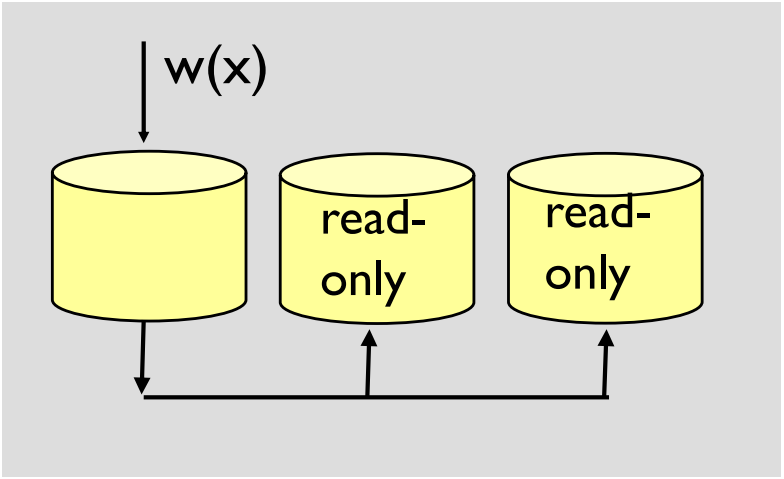


Where can updates be submitted?

- *Update Anywhere:*
  - Updates can be submitted to any ONE site
  - Site forwards updates to other sites



- *Primary Copy:*
  - Updates can only be submitted to one primary copy (or others forwards requests to primary)
  - Primary forwards updates to secondaries

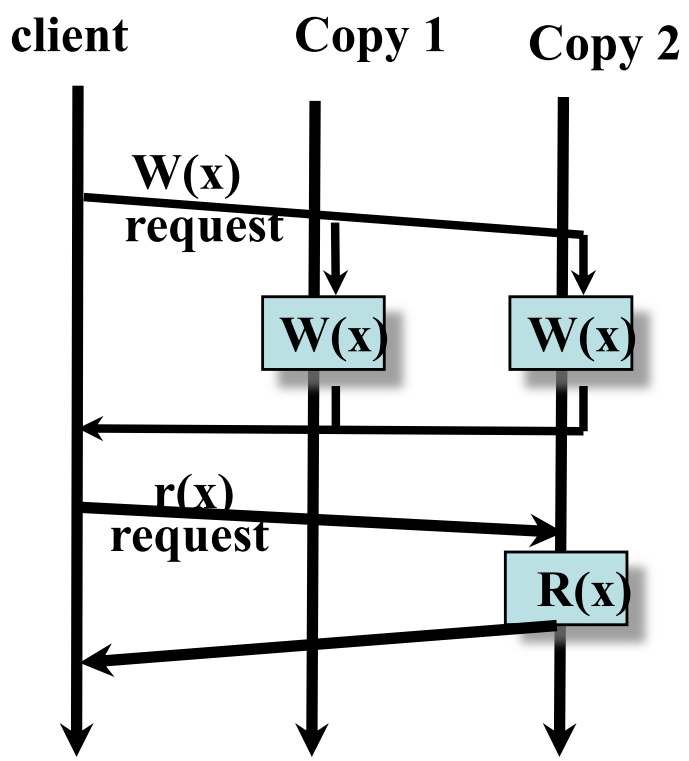


Primary vs. Update anywhere

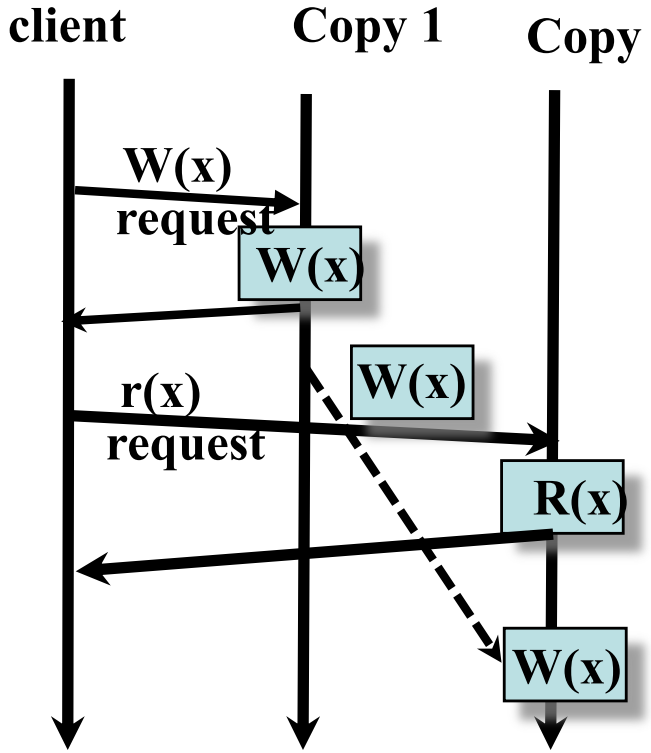
- Pros and Cons?

When to propagate

- *Eager:*
  - within the boundaries of the request
  - Strong Consistency

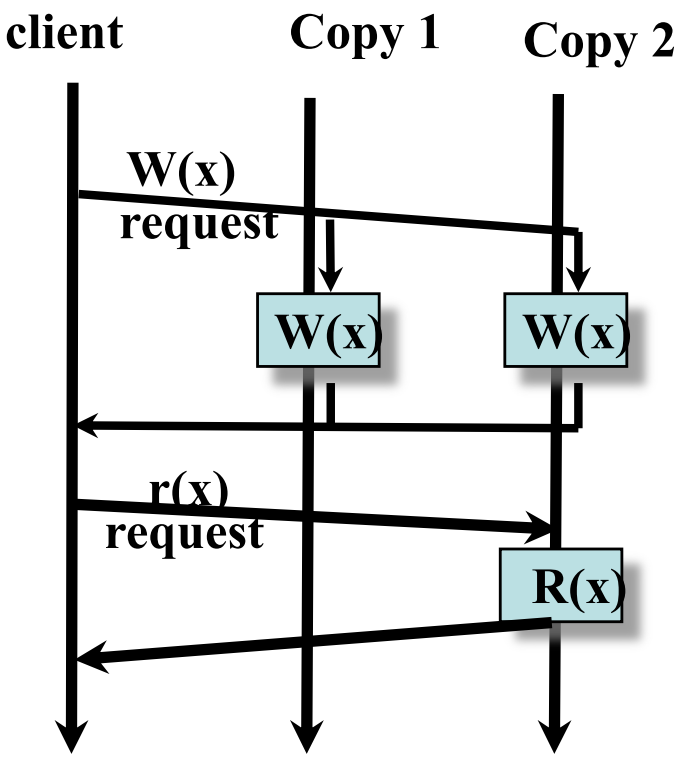


- *Lazy:*
  - After terminating request execution
  - Weak Consistency

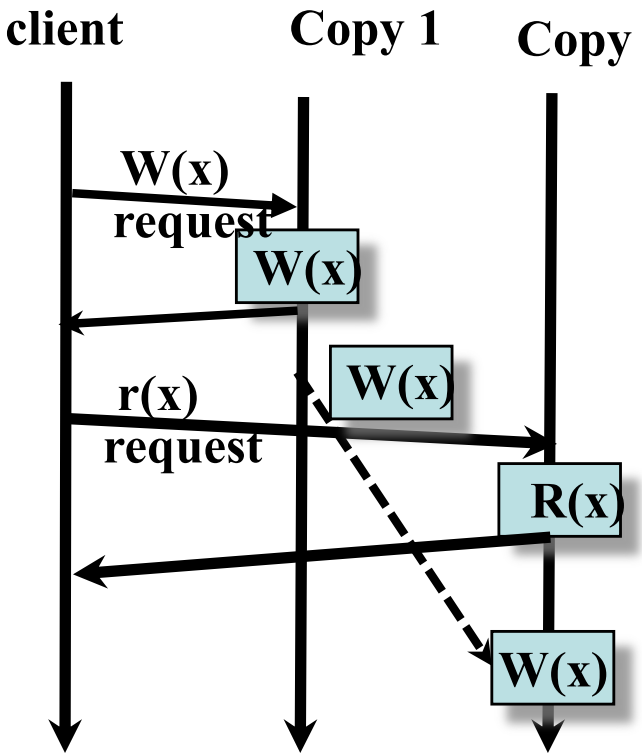


Data Consistency

- Strong (session) consistency
  - Client writes X
  - Updates occurs everywhere
  - Client reads x
  - Gets latest value



- Weak consistency
  - E.g. stale, outdated reads
  - Conflicting updates



Paxos?

Lazy / Update Anywhere

- Conflict detection
- Conflict resolution: eventual consistency
  - Copies should be "eventually" consistent (if there are no further updates for sufficiently long)
    - Site priority
    - value priority
    - earliest/latest timestamp

Lazy vs. Eager

- Lazy vs. Eager: in general

- Combinations
  - Lazy Primary vs. Eager Primary
  - Lazy Update Anywhere vs. Eager Update Anywhere

Examples

- ?

Fault-tolerance vs. Availability

- ***Fault-tolerance***
  - If failure occurs "failover" to other replica
- ***Availability***
  - Service available as long as client can access one replica
- Difference:
  - Fault-tolerance often associated with stronger consistency (e.g., all data copies are always up-to-date)
  - Availability often associated with weaker consistency (might read outdated data if up-to-date data not reachable)
  - Terms not always used clearly

CAP Theorem

- **C**onsistency – **A**vailability – Network **P**artitions

In the Presence of Network Partitions, it is impossible to achieve both (strong) consistency and availability

- Network Partition important in WANs